

In **MySQL**, a **CTE (Common Table Expression)** is a **temporary named result set** that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.

It makes complex queries **easier to read and manage**.

Syntax of CTE

```
WITH cte_name AS (  
    SELECT column1, column2, ...  
    FROM table_name  
    WHERE condition  
)  
SELECT *  
FROM cte_name;
```

Example: Simple (Non-Recursive) CTE

Suppose we have this table:

```
CREATE TABLE employees (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees VALUES  
(1, 'John', 1, 5000),  
(2, 'Sara', 1, 6000),  
(3, 'Mike', 2, 5500),  
(4, 'Emma', 2, 7000),  
(5, 'Tom', 3, 4000);
```

Using a CTE:

```
WITH high_salary AS (  
    SELECT emp_name, dept_id, salary
```

```

FROM employees
WHERE salary > 5500
)
SELECT * FROM high_salary;

```

Output:

emp_name	dept_id	salary
Sara	1	6000
Emma	2	7000

In **MySQL**, a **Window Function** is used to perform **calculations across a set of rows related to the current row, without grouping** the data into a single output row like GROUP BY does.

Syntax

```

function_name(expression)
OVER (
    [PARTITION BY column_name]
    [ORDER BY column_name]
)

```

Explanation:

- **function_name** → The window function (e.g. ROW_NUMBER(), RANK(), SUM())
- **PARTITION BY** → Divides the result set into groups (like GROUP BY, but keeps all rows visible)
- **ORDER BY** → Defines the order of rows for calculations (like ranking or running totals)

Example Table

```

CREATE TABLE sales (
    id INT,
    employee VARCHAR(50),
    department VARCHAR(50),
    sales_amount DECIMAL(10,2)
);

```

```
INSERT INTO sales VALUES
```

```
(1, 'John', 'A', 500),  
(2, 'Sara', 'A', 700),  
(3, 'Mike', 'B', 400),  
(4, 'Emma', 'B', 800),  
(5, 'Tom', 'A', 600);
```

Example: ROW_NUMBER() — Unique Row Number per Group

```
SELECT
```

```
    employee,  
    department,  
    sales_amount,
```

```
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY sales_amount DESC) AS  
rank_in_dept
```

```
FROM sales;
```

Result:

employee	department	sales_amount	rank_in_dept
Sara	A	700	1
Tom	A	600	2
John	A	500	3
Emma	B	800	1
Mike	B	400	2